

Estructuras de datos

Manual operativo de datos y esquemas JSON

Objetivo de la estructura de datos

El sistema utiliza **Airtable** como **memoria central** del ecosistema de automatización.

La base se denomina **Base Coder** y está compuesta por tres tablas principales:

Consultas

Almacena cada consulta recibida por Gmail y su evolución durante el procesamiento.

Conocimiento

Funciona como base de conocimiento del gimnasio. Contiene la información autorizada que el modelo de IA puede utilizar para redactar respuestas.

Log Ejecuciones

Registra el resultado de las ejecuciones del flujo, incluyendo procesos exitosos, revisiones humanas y errores.

La estructura permite mantener separados:

Datos operativos → **Conocimiento** → **Auditoría**

Además, las tablas se encuentran relacionadas para evitar información aislada y permitir trazabilidad entre una consulta y las ejecuciones asociadas.

Tabla Consultas

La tabla **Consultas** constituye el registro principal de cada interacción con un cliente.

Cuando Gmail detecta una nueva consulta válida, n8n crea inmediatamente un registro con estado inicial **Pendiente**.

Campo	Tipo	Función
ID Consulta	Número/autonumérico	Identificador interno de la consulta
Fecha	Fecha y hora	Momento de recepción del correo
Nombre	Texto	Nombre extraído del remitente
Email	Texto	Dirección de correo del cliente
Asunto	Texto	Asunto del mensaje recibido
Mensaje	Texto largo	Contenido de la consulta
Intención	Texto	Categoría detectada por GPT
Confianza	Número	Nivel de confianza de clasificación, entre 0 y 1
Respuesta IA	Texto largo	Respuesta propuesta por el modelo
Requiere Humano	Booleano	Indica si debe activarse HITL
Aprobado	Booleano	Resultado de la aprobación humana
Estado	Selección	Estado actual del procesamiento
Tema Conocimiento	Relación	Vínculo con la base de conocimiento
Error	Texto	Descripción de una falla, cuando corresponde
Gmail Thread ID	Texto	Identificador del hilo de Gmail
Log Ejecuciones	Relación	Registros de auditoría vinculados

Estados utilizados

El ciclo de vida contempla los siguientes estados:

Pendiente
Aprobado
Respondido
Revisión manual
Error

En la implementación actual, una consulta se crea inicialmente como:

```
{
  "Estado": "Pendiente",
  "Requiere Humano": false,
  "Confianza": 0
}
```

El resto de los campos son completados dinámicamente a medida que avanza la ejecución.

Tabla Conocimiento

La tabla **Conocimiento** funciona como la fuente de información autorizada del sistema.

Su objetivo es evitar que el LLM responda utilizando información inventada o conocimiento externo.

Campo	Tipo	Función
Tema	Texto	Nombre específico del conocimiento
Categoría	Texto	Categoría asociada a la intención
Información	Texto largo	Información que puede utilizar la IA
Restricción	Texto largo	Límites o reglas que debe respetar
Activo	Booleano	Determina si el registro puede utilizarse
Consultas	Relación	Consultas vinculadas al conocimiento
Consultas 2	Relación auxiliar	Relación generada durante la construcción de la base

Las categorías actualmente contempladas son:

HORARIOS
PRECIOS
ACTIVIDADES
SEDES
BAJA
COBRO
QUEJA
OTRO
RECLAMO

Ejemplo conceptual:

```
{  
  "Tema": "Horarios generales",  
  "Categoría": "HORARIOS",  
  "Información": "El gimnasio abre de lunes a viernes de 7:00 a 23:00 y los sábados de 8:00  
a 20:00.",  
  "Restricción": "Los horarios de feriados se informan por separado.",  
  "Activo": true  
}
```

n8n recupera los registros de esta tabla y posteriormente filtra aquellos cuya **Categoría** coincide con la intención clasificada y cuyo campo **Activo** es verdadero.

La lógica utilizada es equivalente a:

Categoría del conocimiento = intención detectada
AND
Activo = true

De esta manera, el modelo generativo recibe solamente información relevante y vigente.

Tabla Log Ejecuciones

La tabla **Log Ejecuciones** proporciona trazabilidad y permite construir el dashboard de control.

Cada resultado relevante del workflow genera un registro.

Campo	Tipo	Función
Fecha	Fecha/hora	Momento de generación del log
Consulta	Relación	Consulta asociada
Resultado	Selección	Resultado de la ejecución
Nodo	Selección	Etapa que generó el registro
Detalle Error	Texto	Descripción del error
Requirió HITL	Booleano	Indica intervención humana
Aprobado	Booleano	Resultado de HITL
Duración	Número	Campo reservado para duración de ejecución

Resultados posibles

OK

ERROR

REVISIÓN MANUAL

Nodos registrados

Entre otros:

GPT - Clasificar y responder

Gmail - Aprobación HITL

Validación de entrada

Ejemplo de ejecución automática correcta:

```
{
  "Resultado": "OK",
  "Nodo": "GPT - Clasificar y responder",
  "Requirió HITL": false,
  "Aprobado": false
}
```

Ejemplo de intervención humana aprobada:

```
{  
  "Resultado": "OK",  
  "Nodo": "Gmail - Aprobación HITL",  
  "Requirió HITL": true,  
  "Aprobado": true  
}
```

Ejemplo de rechazo:

```
{  
  "Resultado": "REVISIÓN MANUAL",  
  "Nodo": "Gmail - Aprobación HITL",  
  "Requirió HITL": true,  
  "Aprobado": false  
}
```

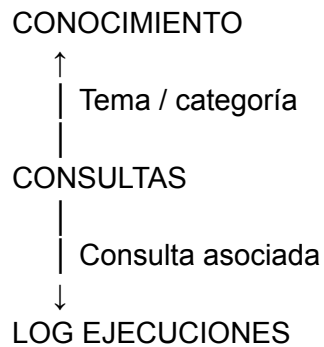
Y ante una falla de GPT:

```
{  
  "Resultado": "ERROR",  
  "Nodo": "GPT - Clasificar y responder",  
  "Detalle Error": "Error en GPT - Clasificación",  
  "Requirió HITL": false,  
  "Aprobado": false  
}
```

Estas rutas están implementadas explícitamente en el workflow exportado.

Relaciones entre las tablas

La estructura puede representarse de la siguiente manera:



Consultas ↔ Conocimiento

Permite identificar qué información de negocio está relacionada con una determinada consulta.

El modelo primero determina la intención y luego n8n busca información cuya categoría coincida.

Ejemplo:

Consulta:

"¿A qué hora abre el gimnasio?"

↓

Intención:

HORARIOS

↓

Conocimiento:

Horarios generales

Consultas ↔ Log Ejecuciones

Cada registro de **Log Ejecuciones** puede vincularse con el registro original de **Consultas**.

Esto permite reconstruir qué ocurrió con una determinada interacción:

Consulta #24

↓

Clasificación GPT

↓

Requirió HITL

↓

Aprobación humana

↓

Respuesta enviada

↓

Log = OK

Esquemas JSON de transferencia

A. Gmail → n8n

El Gmail Trigger recibe información del correo original.

Los principales datos utilizados por el workflow son:

```
{
  "id": "ID_MENSAJE_GMAIL",
  "threadId": "ID_HILO_GMAIL",
  "From": "Nombre Cliente <cliente@email.com>",
  "Subject": "Consulta gimnasio",
  "snippet": "Quiero saber cuáles son los horarios del gimnasio.",
  "internalDate": 1786313688000
}
```

Uso de cada variable

id

→ permite responder mediante Gmail Reply.

threadId

→ se conserva en Airtable para trazabilidad.

From

→ se utiliza para extraer nombre y email.

Subject

→ asunto de la consulta.

snippet

→ mensaje enviado al modelo de IA.

internalDate

→ se transforma a fecha ISO para Airtable.

El workflow incluye además un filtro anti-loop:

in:inbox -from:luкас.tarantostevanovich@gmail.com

De esta forma, los mensajes enviados por la propia cuenta automatizada no vuelven a activar el proceso.

JSON de clasificación de intención

El primer modelo GPT no redacta la respuesta.

Su función es exclusivamente clasificar la consulta.

El resultado solicitado tiene exactamente la siguiente estructura:

```
{  
  "intencion": "HORARIOS",  
  "confianza": 0.95,  
  "requiere_humano": false,  
  "tema_conocimiento": "Horarios generales"  
}
```

Tipos de datos

intencion → string

confianza → number

requiere_humano → boolean

tema_conocimiento → string

El workflow convierte posteriormente la salida del modelo mediante el nodo:

Parsear clasificación GPT

utilizando **JSON.parse()** para transformar la respuesta de texto del LLM en datos estructurados utilizables por n8n.

Reglas de clasificación

El sistema admite una única intención por consulta:

HORARIOS
PRECIOS
ACTIVIDADES
SEDES
BAJA
RECLAMO
QUEJA
COBRO
OTRO

Las siguientes intenciones siempre requieren intervención humana:

BAJA
RECLAMO
QUEJA
COBRO
OTRO

Además, cuando el modelo no puede determinar la intención con suficiente seguridad debe utilizar:

```
{  
  "intencion": "OTRO",  
  "requiere_humano": true  
}
```

Esto evita que una consulta ambigua termine generando automáticamente una acción sensible.

Airtable → GPT: contexto de conocimiento

Una vez clasificada la consulta, n8n recupera los registros activos de **Conocimiento**.

El objeto relevante tiene conceptualmente esta estructura:

```
{
  "id": "recXXXXXXXX",
  "fields": {
    "Tema": "Horarios generales",
    "Categoría": "HORARIOS",
    "Información": "El gimnasio abre de lunes a viernes de 7:00 a 23:00 y los sábados de 8:00 a 20:00.",
    "Restricción": "Los horarios de feriados se informan por separado.",
    "Activo": true
  }
}
```

El segundo modelo recibe principalmente:

Nombre del cliente
Consulta original
Información
Restricción

El prompt establece expresamente que debe utilizar **exclusivamente la información proporcionada por la base**, sin inventar datos ni utilizar conocimiento externo.

GPT → Gmail

La segunda llamada al modelo genera un texto destinado al cliente.

Ejemplo conceptual:

```
{  
  "text": "Hola Lucas, el gimnasio abre de lunes a viernes de 7:00 a 23:00 y los sábados de 8:00 a 20:00."  
}
```

El texto se extrae de la respuesta del nodo OpenAI y se entrega posteriormente al nodo Gmail.

Las respuestas se envían mediante la operación:

Message → Reply

utilizando:

Message ID = ID del mensaje recibido originalmente

Esto permite conservar la respuesta dentro del mismo hilo de conversación de Gmail.

JSON del proceso Human-in-the-loop

Cuando **requiere_humano = true**, el sistema no contacta inmediatamente al cliente.

Primero envía una solicitud de aprobación que contiene:

```
{  
  "cliente": "Lucas Taranto",  
  "email": "cliente@email.com",  
  "consulta": "Me cobraron dos veces la cuota.",  
  "intencion": "COBRO",  
  "confianza": 0.95,  
  "respuesta_propuesta": "Texto generado por IA"  
}
```

El operador puede:

Approve

o

Decline

La respuesta del nodo se procesa mediante:

```
{  
  "data": {  
    "approved": true  
  }  
}
```

Si:

approved = true

se envía la respuesta al cliente y la consulta termina como:

Estado = Respondido

Aprobado = true

Si:

approved = false

no se envía la respuesta y el registro termina como:

Estado = Revisión manual

Aprobado = false

La lógica está implementada mediante el nodo **¿Aprobado por humano?**.

Transferencia de errores

El sistema contempla dos errores principales.

Datos incompletos

Si el mensaje no contiene información válida para continuar, se registra:

```
{
  "Resultado": "ERROR",
  "Nodo": "Validación de entrada",
  "Detalle Error": "Datos incompletos: falta remitente o mensaje.",
  "Requirió HITL": false,
  "Aprobado": false
}
```

Error del modelo de IA

El nodo **Clasificar intención** está configurado para continuar mediante su **salida de error** en lugar de detener completamente el workflow.

La consulta se actualiza a:

```
{
  "Estado": "Error",
  "Confianza": 0,
  "Requiere Humano": false,
  "Aprobado": false,
  "Error": "Error en GPT - Clasificación"
}
```

Y paralelamente se crea un registro en **Log Ejecuciones**:

```
{
  "Resultado": "ERROR",
  "Nodo": "GPT - Clasificar y responder",
  "Detalle Error": "Error en GPT - Clasificación"
}
```